SOFTWARE ARCHITECTURE

Organized Study Notes • Concepts, Architecture Styles, and Trade-offs

Definition
Software architecture is the high-level structure of a software system. It organizes the code into components and defines how those components communicate and depend on one another.

Why architecture matters

- Makes the system easier to understand and organize.
- Helps teams change, test, and maintain the system.
- Supports scaling and clarifies responsibilities between components.

A good architecture should make software easier to change, easier to test, and easier to scale.

1. Common Architecture Styles

Style	Core idea	Strengths	Limitations / notes
N-Tier / Layered	Separates the application into layers, commonly Presentation (UI), Business Logic, Data Access, and Database.	Clear responsibilities; familiar structure; can be maintainable.	Layers can add latency. A shared database or tightly coupled layers can limit independent scaling.
Microservices	Splits the system into small, independently deployable services, often organized around business capabilities.	Independent deployment and scaling; teams can work on services separately.	More operational complexity; network failures, monitoring, and data consistency need attention.
Clean / Onion Architecture	Keeps domain/business rules at the center and makes outer infrastructure depend on inner abstractions; dependencies point inward.	Strong domain isolation; testable design; infrastructure is easier to replace.	Requires discipline and abstractions; may feel excessive for very small applications.
Peer-to-Peer (P2P)	Nodes can act as both clients and servers and communicate directly.	Can distribute workload and reduce dependence on one central server.	Discovery, security, reliability, and coordination can be difficult.

Quick memory aid

- N-Tier: organize by technical layer.
- Microservices: split by business capability into independently deployable services.
- Clean / Onion: protect the domain; dependencies point inward.
- P2P: peers can act as both clients and servers.

2. Criteria for Choosing an Architecture

Testability

How easily can components be tested in isolation? Clear boundaries and dependency injection often help.

Maintainability & Modularity

Can a change be made in one module without causing unrelated changes elsewhere?

Flexibility

How easily can technologies or components be replaced without rewriting the whole application?

Scalability

Can the system handle more users, requests, or data as demand grows?

Complexity & Performance

Does the design add operational or communication overhead? Is the performance acceptable?

Important separation of concerns

Keep the following responsibilities distinct so each can change with fewer side effects:

Presentation / UI	Displays information and receives user input.
Business Logic	Implements business rules and decisions.
Data Access	Reads and writes data through repositories, queries, or other persistence abstractions.
Database / Infrastructure	Stores data and provides technical services such as persistence or external integrations.

Typical layered flow

End User → Presentation / UI → Business Logic → Data Access → Database

This is a common example, not a rule that every architecture must follow exactly.

3. N-Tier vs. Clean / Onion

Aspect	N-Tier / Layered	Clean / Onion
Main focus	Separating code by technical responsibility.	Protecting domain rules and controlling dependency direction.
Structure	Layers such as UI, business, and data access.	Inner domain/application core with outer adapters and infrastructure.
Dependency rule	Often flows from upper layers to lower layers, depending on implementation.	Source-code dependencies point inward toward the core.
Testing	Can be testable when layers are well separated.	Designed to make core business rules testable without infrastructure.
Scaling	A traditional layered deployment may scale as one unit.	Does not automatically provide distributed scaling; deployment strategy is separate.

4. Limitations of N-Tier Architecture

- Scalability limits: if all layers are deployed together, scaling one busy part may require scaling the whole application.
- Latency / performance: requests may pass through multiple layers, adding overhead if the design is inefficient.
- Shared database bottleneck: many components may compete for the same database; database capacity can become the limiting factor.
- Change propagation: poorly designed layers can become tightly coupled, so a change in one layer affects others.

These are possible limitations, not inevitable problems. Caching, optimized queries, clear boundaries, and independent deployment where appropriate can help.

5. Exam-Ready Summary

- Architecture defines the high-level structure of a software system and the interactions among its components.
- Good architecture aims for maintainability, testability, flexibility, scalability, and acceptable performance.
- N-Tier separates technical responsibilities into layers.
- Microservices split a system into small, independently deployable services.
- Clean / Onion architecture keeps domain rules independent of infrastructure, with dependencies pointing inward.
- P2P allows peers to act as both clients and servers.
- Architecture is a trade-off: improving one quality attribute may increase cost, complexity, or latency elsewhere.

Note: The summary keeps the ideas from the handwritten notes and adds a few clarifications where the original wording could be misleading.